Dynatrace Workshop: Kubernetes KIND Cluster Monitoring

Hands-On Workshop - Setup KIND + Dynatrace + Create Dashboards

---

Workshop Overview

What You'll Learn

- Set up a local Kubernetes cluster using KIND (Kubernetes IN Docker)
- Install Dynatrace OneAgent Operator on Kubernetes
- Deploy sample microservices application
- Create comprehensive Kubernetes monitoring dashboards in Dynatrace
- Configure alerts for pod failures and resource issues

Prerequisites

- Computer with Docker installed (Windows/Mac/Linux)
- 8GB RAM minimum, 16GB recommended
- Dynatrace trial account (free 15-day trial)
- Basic understanding of Kubernetes concepts

Workshop Duration

3-4 hours

Part 1: Environment Setup (45 minutes)

Step 1: Install Docker Desktop

For Windows:

1. Download Docker Desktop from: https://www.docker.com/products/docker-desktop/
2. Run the installer
3. Follow installation wizard
4. Restart your computer
5. Start Docker Desktop
6. Wait for Docker to be running (green icon in system tray)

For Mac:

1. Download Docker Desktop for Mac
2. Open the .dmg file
3. Drag Docker to Applications
4. Start Docker Desktop
5. Wait for initialization

For Linux:

1. Follow instructions at: https://docs.docker.com/engine/install/
2. Start Docker service: `sudo systemctl start docker`

Verify Docker is running:

1. Open terminal/command prompt
2. Run: `docker --version`
3. You should see version information

Step 2: Install kubectl (Kubernetes CLI)

For Windows:

1. Download kubectl from: https://kubernetes.io/docs/tasks/tools/install-kubectl-windows/
2. Or use Chocolatey: `choco install kubernetes-cli`
3. Verify: `kubectl version --client`

For Mac:

1. Using Homebrew: `brew install kubectl`
2. Verify: `kubectl version --client`

For Linux:

1. Run:

```bash
curl -LO "https://dl.k8s.io/release/$(curl -L -s https://dl.k8s.io/release/stable.txt)/bin/linux/amd64/kubectl"
sudo install -o root -g root -m 0755 kubectl /usr/local/bin/kubectl
```

2. Verify: `kubectl version --client`

Step 3: Install KIND (Kubernetes in Docker)

For Windows:

1. Download KIND from: https://kind.sigs.k8s.io/docs/user/quick-start/#installation
2. Or use Chocolatey: `choco install kind`

For Mac:

```bash
brew install kind
```

For Linux:

```bash
curl -Lo ./kind https://kind.sigs.k8s.io/dl/v0.20.0/kind-linux-amd64
chmod +x ./kind
sudo mv ./kind /usr/local/bin/kind
```

Verify KIND installation:

```bash
kind version
```

Step 4: Create KIND Cluster

1. Open terminal/command prompt
2. Create a cluster configuration file named `kind-config.yaml`:

```yaml
kind: Cluster
apiVersion: kind.x-k8s.io/v1alpha4
name: dynatrace-workshop
nodes:
- role: control-plane
  extraPortMappings:
  - containerPort: 30080
    hostPort: 30080
    protocol: TCP
- role: worker
- role: worker
```

3. Save this file to your current directory
4. Create the cluster:

```bash
kind create cluster --config kind-config.yaml
```

5. Wait 2-3 minutes for cluster creation
6. Verify cluster is running:

```bash
kubectl cluster-info --context kind-dynatrace-workshop
kubectl get nodes
```

You should see 3 nodes: 1 control-plane, 2 workers

Step 5: Verify Kubernetes Access

1. Check cluster status:

```bash
kubectl get nodes
```

Output should show:

```
NAME                                STATUS   ROLES           AGE   VERSION
dynatrace-workshop-control-plane   Ready    control-plane   2m    v1.27.3
dynatrace-workshop-worker          Ready    <none>          2m    v1.27.3
dynatrace-workshop-worker2         Ready    <none>          2m    v1.27.3
```

2. Check system pods:

```bash
kubectl get pods -n kube-system
```

All pods should be in "Running" status

Part 2: Dynatrace Account Setup (20 minutes)

Step 6: Create Dynatrace Trial Account

1. Go to: https://www.dynatrace.com/trial/
2. Click **"Start free trial"**
3. Fill in registration:

- Email
- Name
- Company: "Workshop Training"
- Region: Choose closest

4. Click **"Start your free trial"**
5. Verify email and set password
6. Log in to your Dynatrace environment

Your environment URL: `https://XXXXX.live.dynatrace.com`

Step 7: Generate Dynatrace Access Tokens

PaaS Token (for OneAgent Operator):

1. In Dynatrace, click **Settings** (gear icon, bottom-left)
2. Go to **Access tokens**
3. Click **"Generate new token"**
4. Configure:
   - **Token name:** `Kubernetes-OneAgent-PaaS`
   - Enable these scopes:
     - ☑ **PaaS integration - Installer download**
     - ☑ **Access problem and event feed, metrics, and topology**
5. Click **"Generate token"**
6. **COPY THE TOKEN** and save it in a text file
7. Label it: `PAAS_TOKEN`

API Token (for Kubernetes monitoring):

1. Click **"Generate new token"** again
2. Configure:
   - **Token name:** `Kubernetes-API-Access`
   - Enable these scopes:
     - ☑ **Read metrics**
     - ☑ **Ingest metrics**
     - ☑ **Read entities**
     - ☑ **Read settings**
     - ☑ **Write settings**

3. Click **"Generate token"**
4. **COPY THE TOKEN** and save it
5. Label it: `API_TOKEN`

Step 8: Get Dynatrace Environment ID

1. Look at your Dynatrace URL: `https://abc12345.live.dynatrace.com`
2. The environment ID is: `abc12345`
3. Save this - you'll need it

Alternatively:

1. In Dynatrace, go to **Deployment status**
2. Look for **"Environment ID"**
3. Copy the ID

Part 3: Install Dynatrace OneAgent Operator on Kubernetes (30 minutes)

Step 9: Create Dynatrace Namespace

1. Create namespace for Dynatrace:

```bash
kubectl create namespace dynatrace
```

2. Verify namespace created:

```bash
kubectl get namespaces
```

Step 10: Create Dynatrace Secret

Replace the placeholders with your actual values:

```bash
kubectl -n dynatrace create secret generic dynakube \
  --from-literal="apiToken=YOUR_API_TOKEN" \
  --from-literal="paasToken=YOUR_PAAS_TOKEN"
```

Example:

```bash
kubectl -n dynatrace create secret generic dynakube \
  --from-literal="apiToken=dt0c01.XXXXX.YYYYY" \
  --from-literal="paasToken=dt0c01.ZZZZZ.AAAAA"
```

Verify secret created:

```bash
kubectl get secret -n dynatrace
```

Step 11: Install Dynatrace Operator

1. Add Dynatrace Helm repository:

```bash
kubectl apply -f https://github.com/Dynatrace/dynatrace-operator/releases/latest/download/kubernetes.yaml
```

2. Wait for operator to be ready:

```bash
kubectl -n dynatrace wait pod --for=condition=ready --selector=app.kubernetes.io/name=dynatrace-operator,app.kubernetes.io
```

3. Verify operator is running:

```bash
kubectl get pods -n dynatrace
```

You should see dynatrace-operator pods in "Running" status

Step 12: Create DynaKube Custom Resource

1. Create a file named `dynakube.yaml`:

```yaml
```

```
apiVersion: dynatrace.com/v1beta2
kind: DynaKube
metadata:
  name: dynakube
  namespace: dynatrace
spec:
  apiUrl: https://YOUR_ENVIRONMENT_ID.live.dynatrace.com/api
  oneAgent:
    classicFullStack:
      tolerations:
      - effect: NoSchedule
        key: node-role.kubernetes.io/master
        operator: Exists
      - effect: NoSchedule
        key: node-role.kubernetes.io/control-plane
        operator: Exists
```

2. **IMPORTANT:** Replace `YOUR_ENVIRONMENT_ID` with your actual environment ID

Example:

```
yaml

apiUrl: https://abc12345.live.dynatrace.com/api
```

3. Apply the DynaKube:

```
bash

kubectl apply -f dynakube.yaml
```

4. Wait for OneAgent to deploy (takes 3-5 minutes):

```
bash

kubectl get pods -n dynatrace -w
```

5. Check DynaKube status:

```
bash

kubectl get dynakube -n dynatrace
```

Status should show "Running" after a few minutes

Step 13: Verify Dynatrace Connection

1. In Dynatrace web UI, go to **Infrastructure → Kubernetes**
2. You should see your KIND cluster appear within 5-10 minutes
3. If not visible yet, wait a bit longer - initial connection takes time

---

Part 4: Deploy Sample Microservices Application (30 minutes)

Step 14: Deploy Sample Sock Shop Application

We'll deploy a microservices demo application for monitoring.

1. Create namespace:

```bash
kubectl create namespace sock-shop
```

2. Deploy Sock Shop:

```bash
kubectl apply -f https://raw.githubusercontent.com/microservices-demo/microservices-demo/master/deploy/kubernetes/compl
```

3. Wait for all pods to be running:

```bash
kubectl get pods -n sock-shop -w
```

Press Ctrl+C when all pods show "Running" status

Step 15: Expose Application via NodePort

1. Check the front-end service:

```bash
kubectl get svc -n sock-shop front-end
```

2. The service should already be exposed on NodePort
3. Access the application:

```bash
kubectl port-forward -n sock-shop svc/front-end 8080:80
```

4. Open browser to: http://localhost:8080
5. You should see the Sock Shop application
6. Click around the application to generate some traffic:
   - Browse different sock products
   - Add items to cart
   - View cart
   - Try to place an order

Step 16: Deploy Additional Test Applications

Deploy NGINX:

```bash
kubectl create deployment nginx --image=nginx:latest
kubectl expose deployment nginx --port=80 --type=NodePort
```

Deploy a failing pod (for testing alerts):

```bash
kubectl create deployment failing-app --image=busybox -- sh -c "exit 1"
```

This will continuously fail and restart - perfect for testing alerts!

Verify all deployments:

```bash
kubectl get deployments --all-namespaces
```

Part 5: Explore Kubernetes Data in Dynatrace (20 minutes)

Step 17: Navigate to Kubernetes Overview

1. In Dynatrace, go to **Infrastructure → Kubernetes**
2. Click on your cluster: **dynatrace-workshop**
3. Explore the overview:
   - Cluster health
   - Node status
   - Namespace overview

- Pod statistics

Step 18: Explore Workloads

1. Click on **Workloads** tab
2. You'll see:
   - Deployments
   - StatefulSets
   - DaemonSets
   - Jobs/CronJobs
3. Click on **sock-shop** namespace
4. Explore different microservices:
   - front-end
   - catalogue
   - orders
   - payment
   - etc.

Step 19: Explore Individual Pods

1. Click on any pod (e.g., front-end pod)
2. You'll see:
   - Pod properties
   - CPU and memory usage
   - Container information
   - Events and logs
3. Click **"View logs"** to see container logs

Step 20: Explore Nodes

1. Go back to cluster overview
2. Click on **Nodes** tab
3. Click on a worker node
4. Observe:
   - Node resource usage
   - Running pods
   - Node conditions

- Capacity and allocations

Part 6: Create Kubernetes Dashboard in Dynatrace (60 minutes)

Now we'll create a comprehensive dashboard!

Step 21: Create New Dashboard

1. In Dynatrace, click **Dashboards** from left menu
2. Click **"Create dashboard"**
3. Name: `Kubernetes Cluster Monitoring - Workshop`
4. Click **"Create"**

Step 22: Add Cluster Overview Tile

1. Click **"Add tile"**
2. Select **"Kubernetes"**
3. Configure:
   - **Title:** `Cluster Health Overview`
   - **Cluster:** Select your dynatrace-workshop cluster
   - **Visualization:** Summary
4. Click **"Save changes to dashboard"**

Step 23: Add Node Health Tile

1. Click **"Add tile"**
2. Select **"Metric"**
3. Configure:
   - **Title:** `Node CPU Usage`
   - Click **"Select metric"**
   - Search: `builtin:kubernetes.node.cpu_usage`
   - Select it
4. Under **"Split by":**
   - Select **"Kubernetes node"**
5. Visualization: **Line chart**
6. Click **"Save changes to dashboard"**

Step 24: Add Node Memory Tile

1. Click **"Add tile"**
2. Select **"Metric"**
3. Configure:
   - **Title:** `Node Memory Usage`
   - **Metric:** `builtin:kubernetes.node.memory_working_set`
   - **Split by:** Kubernetes node
   - **Visualization:** Line chart
4. Click **"Save changes to dashboard"**

Step 25: Add Pod Count Tile

1. Click **"Add tile"**
2. Select **"Metric"**
3. Configure:
   - **Title:** `Running Pods by Namespace`
   - **Metric:** `builtin:kubernetes.pods`
   - **Split by:** Kubernetes namespace
   - **Visualization:** Stacked bar chart
4. Click **"Save changes to dashboard"**

Step 26: Add Container Restarts Tile

1. Click **"Add tile"**
2. Select **"Metric"**
3. Configure:
   - **Title:** `Container Restarts (Last 24h)`
   - **Metric:** `builtin:kubernetes.container.restart_count`
   - **Split by:** Kubernetes workload
   - **Visualization:** Table or bar chart
   - **Aggregation:** Sum
4. Click **"Save changes to dashboard"**

This will show your failing-app pod restarting!

Step 27: Add Pod CPU Usage Tile

1. Click **"Add tile"**

2. Select **"Metric"**
3. Configure:
   - **Title:** `Top Pods by CPU Usage`
   - **Metric:** `builtin:kubernetes.pod.cpu_usage`
   - **Split by:** Kubernetes pod
   - **Visualization:** Top list (shows top 10)
4. Click **"Save changes to dashboard"**

Step 28: Add Pod Memory Usage Tile

1. Click **"Add tile"**
2. Select **"Metric"**
3. Configure:
   - **Title:** `Top Pods by Memory Usage`
   - **Metric:** `builtin:kubernetes.pod.memory_working_set`
   - **Split by:** Kubernetes pod
   - **Visualization:** Top list
4. Click **"Save changes to dashboard"**

Step 29: Add Namespace Resource Quota Tile

1. Click **"Add tile"**
2. Select **"Metric"**
3. Configure:
   - **Title:** `Namespace Resource Usage`
   - **Metric:** `builtin:kubernetes.namespace.cpu_usage`
   - **Split by:** Kubernetes namespace
   - **Visualization:** Pie chart
4. Click **"Save changes to dashboard"**

Step 30: Add Problems Tile

1. Click **"Add tile"**
2. Select **"Problems"**
3. Configure:
   - **Title:** `Kubernetes Problems`
   - **Filter:** Add filter for Kubernetes entities
   - **Status:** Open problems

- **Timeframe:** Last 24 hours

4. Click **"Save changes to dashboard"**

Step 31: Add Cluster Capacity Tile

1. Click **"Add tile"**
2. Select **"Markdown"**
3. Add content:

```
markdown

## Cluster Capacity Overview

### Nodes: 3
- 1 Control Plane
- 2 Workers

### Monitoring Status
✅ OneAgent Deployed
✅ All Nodes Connected
✅ Dynatrace Operator Running

### Key Metrics to Watch
- CPU Usage > 80%
- Memory Usage > 85%
- Pod Restart Count > 5
- Failed Pods > 0
```

4. Click **"Save changes to dashboard"**

Step 32: Add Network Traffic Tile

1. Click **"Add tile"**
2. Select **"Metric"**
3. Configure:
   - **Title:** `Network Traffic - Received`
   - **Metric:** `builtin:kubernetes.pod.network.bytes_received`
   - **Split by:** Kubernetes namespace
   - **Visualization:** Area chart
4. Click **"Save changes to dashboard"**

Step 33: Add Workload Status Tile

1. Click **"Add tile"**

2. Select **"Metric"**
3. Configure:
   - **Title:** `Deployment Desired vs Available Pods`
   - **Metric:** `builtin:kubernetes.workload.pods_desired`
   - Add another metric: `builtin:kubernetes.workload.pods_available`
   - **Split by:** Kubernetes workload
   - **Visualization:** Bar chart (grouped)
4. Click **"Save changes to dashboard"**

Step 34: Add Sock Shop Specific Tile

1. Click **"Add tile"**
2. Select **"Metric"**
3. Configure:
   - **Title:** `Sock Shop Application Health`
   - **Metric:** `builtin:kubernetes.pods`
   - **Filter by:** Namespace = sock-shop
   - **Split by:** Kubernetes workload
   - **Visualization:** Single value (count)
4. Click **"Save changes to dashboard"**

Step 35: Arrange Dashboard Layout

1. Drag and drop tiles to organize them logically:
   - **Row 1:** Cluster overview and capacity info
   - **Row 2:** Node CPU and Memory usage
   - **Row 3:** Pod counts and restarts
   - **Row 4:** Top resource consumers
   - **Row 5:** Network and workload status
   - **Row 6:** Problems and Sock Shop health
2. Resize tiles for better visualization

Step 36: Configure Dashboard Settings

1. Click dashboard settings (three dots, top-right)
2. Click **"Settings"**
3. Enable **"Auto-refresh"**

4. Set interval: **2 minutes**
5. Configure **"Default timeframe"**: Last 2 hours
6. Click **"Save"**

Part 7: Configure Kubernetes Alerts (45 minutes)

Step 37: Create Alert for High Pod Restart Count

1. Go to **Settings → Anomaly detection → Metric events**
2. Click **"Add metric event"**
3. Configure:
   - **Summary:** `High Pod Restart Count`
   - **Type:** Metric event
   - **Severity:** Error
4. Metric query:
   - **Metric:** `builtin:kubernetes.container.restart_count`
   - **Aggregation:** Sum
   - **Split by:** Kubernetes pod
5. Alert condition:
   - **Threshold:** `> 5`
   - **Violating samples:** 2
   - **Sliding window:** 10 minutes
6. Event description:

```
Pod Restart Alert
Pod: {dims:dt.entity.cloud_application_instance}
Restart Count: {metricValue}
Action: Investigate pod logs and events
```

7. Click **"Save changes"**

Step 38: Create Alert for High Node CPU

1. Click **"Add metric event"**
2. Configure:
   - **Summary:** `Node CPU Usage Critical`
   - **Severity:** Error

3. Metric query:
   - **Metric:** `builtin:kubernetes.node.cpu_usage`
   - **Aggregation:** Average
   - **Split by:** Kubernetes node
4. Alert condition:
   - **Threshold:** `> 80` (80% CPU)
   - **Violating samples:** 3
   - **Sliding window:** 15 minutes
5. Event description:

```
Node CPU Critical
Node: {dims:dt.entity.kubernetes_node}
CPU Usage: {metricValue}%
Action: Check workload distribution
```

6. Click **"Save changes"**

Step 39: Create Alert for High Node Memory

1. Click **"Add metric event"**
2. Configure:
   - **Summary:** `Node Memory Usage Critical`
   - **Severity:** Error
3. Metric query:
   - **Metric:** `builtin:kubernetes.node.memory_working_set`
   - **Aggregation:** Average
   - **Split by:** Kubernetes node
4. Alert condition:
   - **Threshold:** `> 85` (85% memory)
   - **Violating samples:** 3
   - **Sliding window:** 15 minutes
5. Click **"Save changes"**

Step 40: Create Alert for Pod Failures

1. Click **"Add metric event"**
2. Configure:
   - **Summary:** `Pod in Failed State`

- **Severity:** Error

3. Metric query:
   - **Metric:** `builtin:kubernetes.pods`
   - **Filter:** Phase = Failed
   - **Aggregation:** Count
   - **Split by:** Kubernetes namespace
4. Alert condition:
   - **Threshold:** `> 0`
   - **Violating samples:** 1
   - **Sliding window:** 5 minutes
5. Click **"Save changes"**

Step 41: Create Alerting Profile for Kubernetes

1. Go to **Settings → Alerting → Alerting profiles**
2. Click **"Add alerting profile"**
3. Configure:
   - **Name:** `Kubernetes Alerts`
   - **Management zone:** Select your Kubernetes cluster (if you created one)
4. Severity rules:
   - Include Error and Custom alert severities
   - Add filter: Title contains "Kubernetes" OR "Node" OR "Pod"
5. Click **"Save changes"**

Step 42: Create Email Notification for Kubernetes Alerts

1. Go to **Settings → Integration → Problem notifications**
2. Click **"Add notification"**
3. Select **"Email"**
4. Configure:
   - **Display name:** `Kubernetes Cluster Alerts`
   - **Alerting profile:** Select "Kubernetes Alerts"
   - **Recipients:** Your email address
5. Subject:

```
[Kubernetes Alert] {ProblemTitle}
```

6. Body:

```
Kubernetes Cluster Alert

Problem: {ProblemTitle}
Severity: {ProblemSeverity}
Details: {ProblemDetailsText}

Cluster: dynatrace-workshop
Problem URL: {ProblemURL}

Recommended Actions:
1. Check pod/node status: kubectl get pods --all-namespaces
2. Review logs: kubectl logs <pod-name>
3. Check events: kubectl get events --sort-by='.lastTimestamp'

Time: {State} at {ProblemImpact}
```

7. Click **"Save changes"**

Part 8: Testing Alerts (30 minutes)

Step 43: Test Pod Restart Alert

Your failing-app deployment should already be triggering restarts!

1. Check restart count:

```bash
kubectl get pods | grep failing-app
```

2. You should see RESTARTS column incrementing
3. Go to **Observe and explore → Problems** in Dynatrace
4. Within 10-15 minutes, you should see a problem for "High Pod Restart Count"
5. Check your email for notification

Step 44: Test High CPU Alert (Stress Test)

1. Deploy a CPU stress pod:

```bash
kubectl run cpu-stress --image=polinux/stress --restart=Never -- stress --cpu 2 --timeout 600s
```

This will run for 10 minutes stressing CPU

2. Watch pod CPU:

```
bash

kubectl top pod cpu-stress
```

3. Check dashboard - CPU usage should spike
4. After 15 minutes, check for CPU alert in Problems

Step 45: Test Pod Failure Alert

1. Create a pod that fails:

```
bash

kubectl run test-failure --image=nginx:invalid-tag
```

This will fail to pull the image

2. Check pod status:

```
bash

kubectl get pods test-failure
```

Status should show "ImagePullBackOff" or "Error"

3. Check Dynatrace Problems - should see pod failure alert

Step 46: Verify Dashboard Shows All Issues

1. Go to your Kubernetes dashboard
2. Verify you can see:
   - ✅ Increased restart count for failing-app
   - ✅ High CPU usage from stress test
   - ✅ Failed pod in pod count
   - ✅ Problems tile showing active issues

Part 9: Advanced Dashboard Features (30 minutes)

Step 47: Add Custom Filters to Dashboard

1. Open your dashboard

2. Click dashboard settings → **Settings**
3. Under **"Dashboard variables"**, click **"Add variable"**
4. Configure:
   - **Display name:** `Namespace`
   - **Variable name:** `namespace`
   - **Type:** Entity filter
   - **Filter:** Kubernetes namespace
5. Click **"Save"**
6. Add another variable:
   - **Display name:** `Timeframe`
   - **Variable name:** `timeframe`
   - **Type:** Timeframe picker
7. Click **"Save"**

Now you can filter entire dashboard by namespace!

Step 48: Create Drill-Down Links

1. Edit a tile (e.g., "Running Pods by Namespace")
2. In tile settings, find **"Custom drilldown"**
3. Enable it
4. Set drill-down target: Kubernetes namespace overview
5. Save changes

Now clicking on a namespace in the chart will take you to detailed view!

Step 49: Add Conditional Formatting

1. Edit the "Container Restarts" tile
2. Under **"Visualization"**, select **"Table"**
3. Enable **"Conditional formatting"**
4. Add rules:
   - If restart count > 10: Red
   - If restart count 5-10: Orange
   - If restart count < 5: Green
5. Save changes

Step 50: Clone Dashboard for Different Teams

1. Click dashboard settings → **"Clone"**
2. Name: `Kubernetes - Developer View`
3. Modify for developers:
   - Remove node-level metrics
   - Focus on application pods
   - Add service endpoints
   - Add log tiles
4. Save

Create another clone:

- Name: `Kubernetes - Operations View`
- Focus on infrastructure metrics
- Node health
- Resource allocation
- Cluster capacity

Part 10: Kubernetes Best Practices in Dynatrace (20 minutes)

Step 51: Set Up Resource Quotas Monitoring

1. Check if your namespaces have resource quotas:

```
bash

kubectl get resourcequota --all-namespaces
```

2. Create a dashboard tile monitoring quota usage vs limits

Step 52: Configure Pod Disruption Budget Alerts

1. Create metric event for PDB violations
2. Monitor pod availability during updates

Step 53: Set Up Persistent Volume Monitoring

1. Add dashboard tile for PV usage:
   - Metric: `builtin:kubernetes.pv.capacity`
   - Split by: Persistent Volume

Step 54: Create SLO for Application Availability

1. Go to **Manage → SLOs**
2. Click **"Add SLO"**
3. Configure:
   - **Name:** Sock Shop Availability
   - **Target:** 99.5% availability
   - **Evaluation:** Pod availability for sock-shop namespace
4. Save

Add SLO status tile to dashboard

Part 11: Cleanup (Optional - 10 minutes)

If you want to keep practicing:

- Leave everything running
- Continue exploring Dynatrace features
- Deploy more applications

If you want to clean up:

Delete applications:

```
bash

kubectl delete namespace sock-shop
kubectl delete deployment nginx
kubectl delete pod cpu-stress
kubectl delete pod test-failure
kubectl delete deployment failing-app
```

Delete Dynatrace from cluster (optional):

```
bash

kubectl delete dynakube --all -n dynatrace
kubectl delete namespace dynatrace
```

Delete KIND cluster:

```
bash
```

```
kind delete cluster --name dynatrace-workshop
```

Note: Your Dynatrace trial account remains active for 15 days - you can recreate the cluster anytime!

Workshop Summary

What You Accomplished ✅

1. ✅ Set up local Kubernetes cluster using K